



🎮Unityチーム開発で学ぶ！GitHubとFork (Git GUI)を使ったプロジェクト共有入門



むぎ

2025年11月6日 01:47

...



はじめに

Unityでチーム開発を始めると、「誰かが上書きしてシーンが壊れた！」「最新データがどれかわからない...」といったトラブルは起きがちです。

個人開発でも「プロジェクト壊れちゃった！前のデータに戻したい...」といったこともあるかと思います。

あと、「PC壊れた...作ったゲームのデータ無くなったわ...」このようなこともあるかと思います。(やばい)

そもそも「どうやってUnityのプロジェクトを共有するの...?」「USBでデータ渡せばいいの...?」といった人もいませんか。

そのような悩みを解決するのが **Git + GitHub + Fork (Git GUIツール)** の組み合わせです。

『Fork』を使えば、下記のような難しいコマンドを覚えず、ボタン操作だけでGit管理ができます。

もしGitをコマンド操作しようとする...?

```
// リポジトリをローカルのPCにコピー (Clone)
git clone https://github.com/username/unity-game-project.git
cd unity-game-project

// 新しいブランチの作成と切り替え (Checkout)
git checkout -b feature/player-move

// 変更をGitに記録 (Commit)
git add .
git commit -m "Add player movement feature"

// Gitに記録した内容をGitHubにアップロード (Push)
git push origin feature/player-move

// mainブランチにデータを反映して良いかリクエスト送信 (PullRequest)
これはGitHub上で行うため割愛

// mainブランチにデータ反映(Merge)
git checkout main
git pull origin main
git merge feature/player-move
git push origin main

// リモートの変更を取得 (Fetch)
git fetch origin

// 最新の変更を反映 (Pull)
git pull origin main

// 不要になったブランチを削除 (Delete Branch)
# ローカルブランチ削除
git branch -d feature/player-move
# リモートブランチ削除
git push origin --delete feature/player-move
```

長すぎやろがい！コマンドばかりでよーわからん💧

この記事では、UnityプロジェクトをGitHubで共有し、Forkを使ってチーム開発を進める手順を、初心者向けにわかりやすく解説します。

この記事を見て、チーム開発の運用方法を「完全に理解」しよう！

💡 想定読者：Unityでチーム制作したいorしている人／Git初心者／コマンドが苦手な人

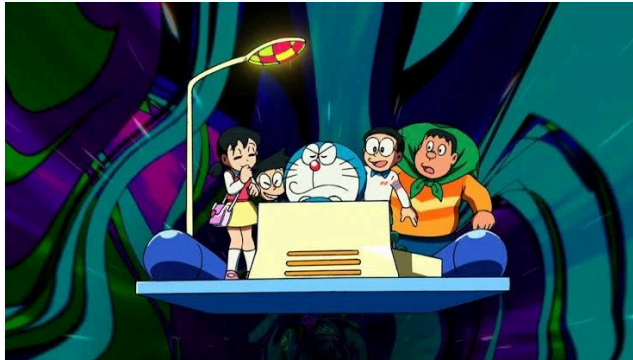
🔧 GitとGitHubの基礎知識

Gitは「ファイルの変更履歴を記録・管理するツール」です。

GitHubは、その履歴をクラウドで共有できるサービス。

公式サイト (<https://github.com/>)

つまり、Gitが「タイムマシン」で、GitHubが「そのデータを保存する倉庫」です。



以前実装していたピー助に会いたい...
タイムマシン (Git) で過去のバージョンに戻ろう...



過去のバージョンに戻ること
で以前実装していたピー助に逢えた...🐍

Unityでは多くのファイルが生成されますが、不要なフォルダを除外する設定 (.gitignore) が重要です。

例えば、『Libraryフォルダ』これは、Unityがプロジェクトを開く際に自動的に作られる**キャッシュフォルダ**です。アセットなどのいろいろな情報が記録されています。

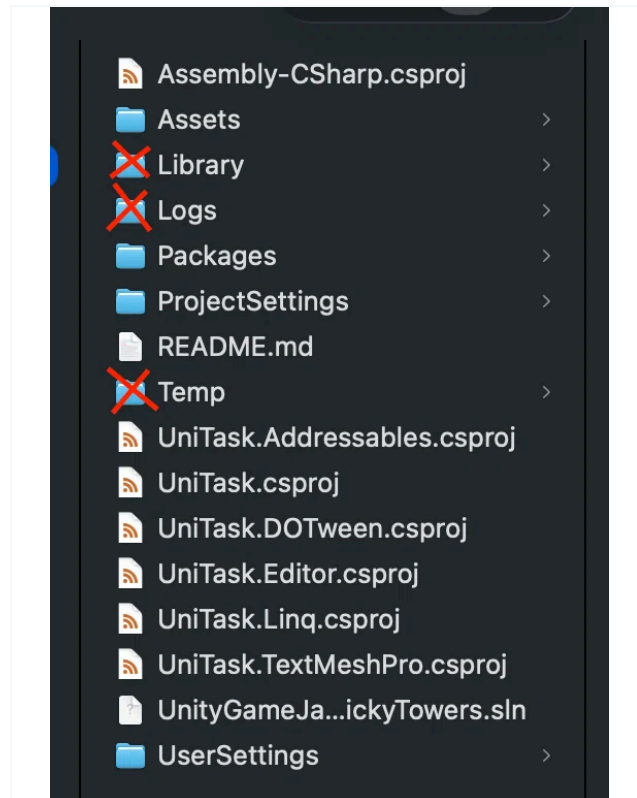
無ければ結局自動生成されるのでいつ削除してもOK。

⚠️ 読み込みが入るため、Unityプロジェクトのロードが長くなる

✅ 推奨：Unity公式の .gitignore テンプレートを使用しましょう。

<https://github.com/github/gitignore/blob/main/Unity.gitignore>

→リポジトリ作る際に.gitignoreは設定できるので、DLしなくてOK👍



Unityプロジェクトのディレクトリ構成。赤バツのフォルダたちはGit管理対象外。
(キャッシュやログなどのデータなので、共有しなくて良いデータ。)

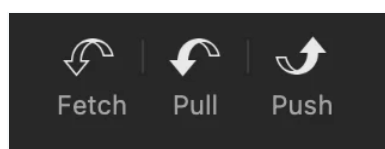
Fork (Git GUIツール)を導入しよう

Forkは、Gitの操作をすべてGUIで行えるアプリです。



フォークのアイコンしているから分かりやすいですね。

コミットやプッシュ、マージなどを「クリックで実行」できます。

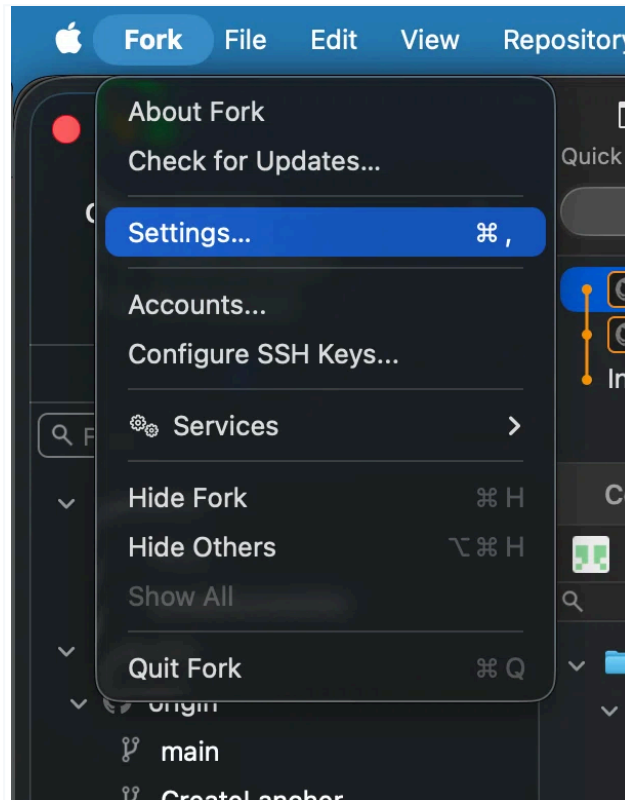


ボタンはこんな感じ。押すだけで実行可能なので楽です。

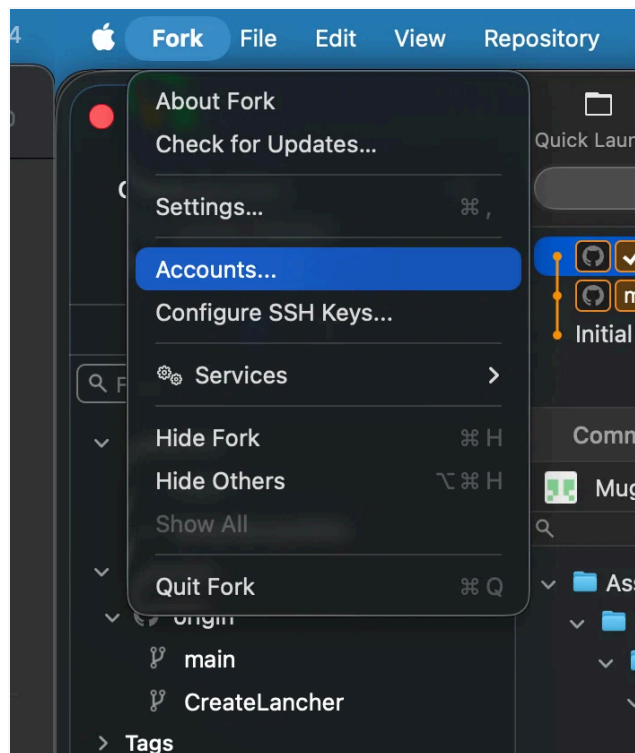
Gitのこのコマンドを覚えて...次このコマンド打って... 「「しんどい」」
スペルミスもめんどくさいですし、視認性が低い。
これを解決しているのがForkというわけです。

公式サイト（<https://git-fork.com>）からダウンロードして、GitHubアカウントと連携しましょう。

初期設定のポイント

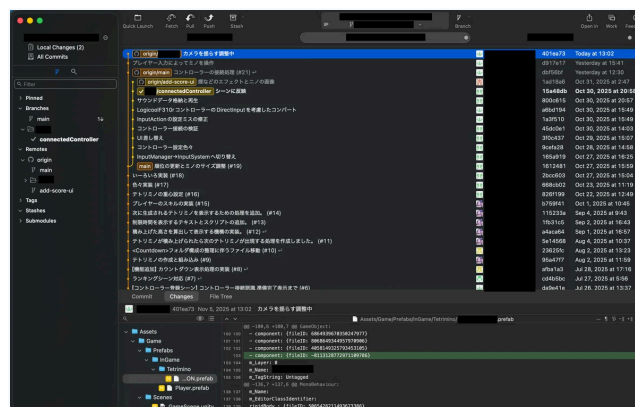


Fork/Settingsから各種設定
リポジトリのディレクトリやツールのアップデート設定などの設定あり。



Fork/AccountsからGitHubアカウント連携

Forkを使えば、プロジェクトの履歴や変更内容を「目で見て」理解できるようになります。

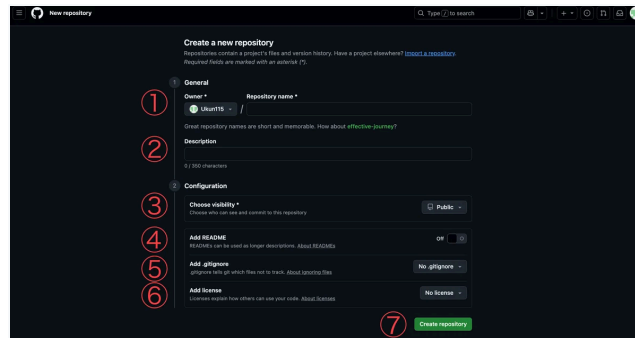
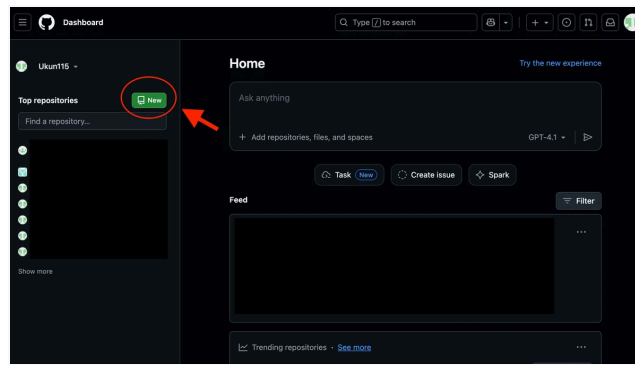


Fork画面

🚀 UnityプロジェクトをGitHubにアップロードする

手順:

1. GitHub (<https://github.com/>) で新規リポジトリを作成



General（一般）

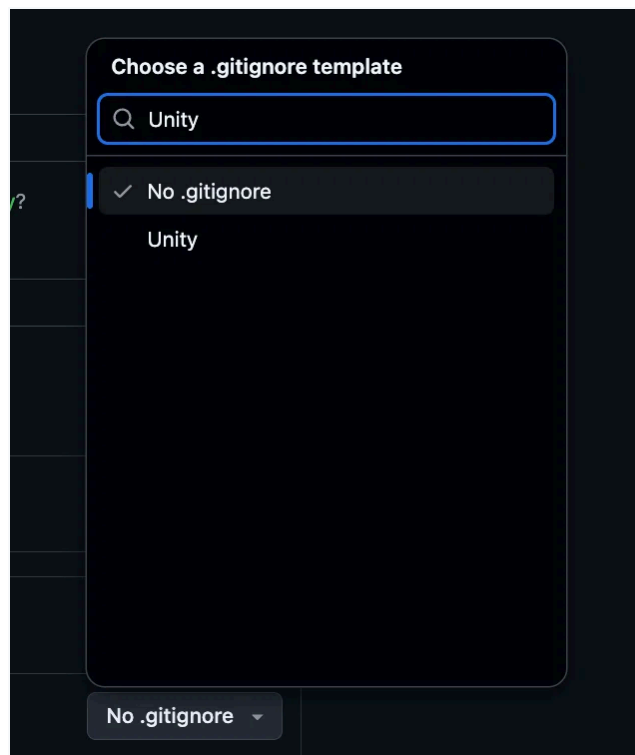
- ①誰がリポジトリの管理者か選択。リポジトリ名を記載。
リポジトリ名はUnityプロジェクトと同名の方が管理しやすいです。

システム上、日本語は文字化けなどの理由から非推奨です⚠
英語で記載しましょう。ローマ字もNGです。

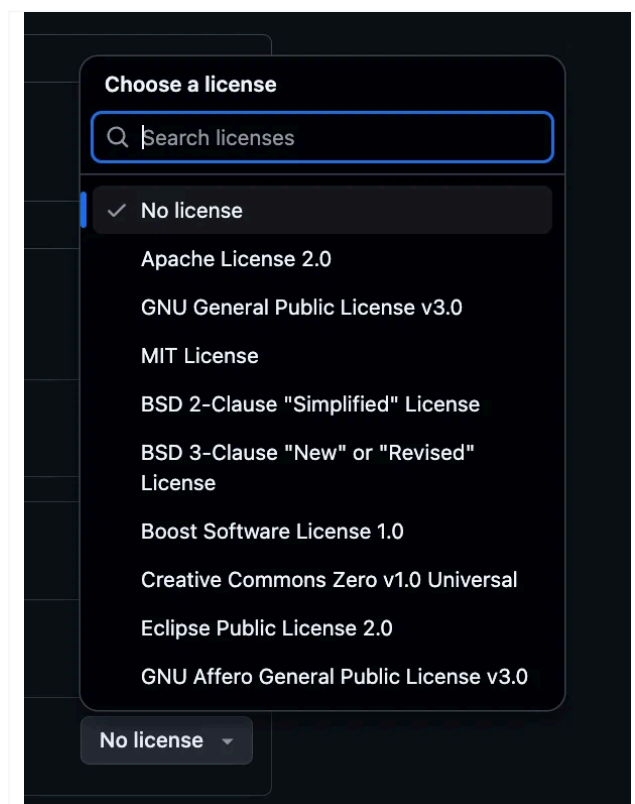
- ②リポジトリの説明。なくてもOK、日本語でOK、です。

Configuration（構成）

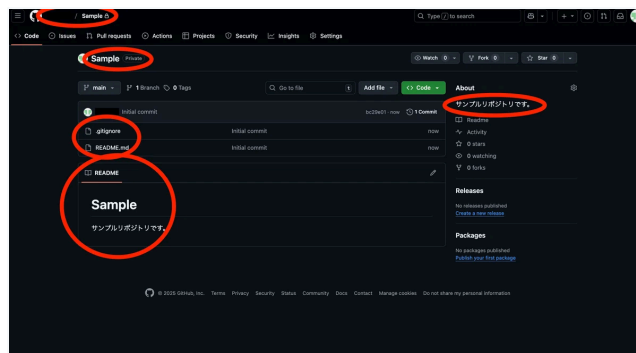
- ③公開範囲設定：誰からでも閲覧やコミットなどアクセス可能でOKならPublic、リポジトリ招待制（非公開）にしたいならPrivateにしましょう。アクセス修飾子（<https://learn.microsoft.com/ja-jp/dotnet/csharp/programming-guide/classes-and-structs/access-modifiers>）と同じですね。
- ④Reame追加：このリポジトリの情報をまとめたファイル。わからなかったらとりあえずoffでOK。あとで手動で追加しましょう。
- ⑤.gitignoreの追加：ボタンを押すとC++やJavaなど、各プロジェクトに適したテンプレート.gitignoreを選ぶことができます。検索インプットフィールドからUnityと検索し選択しましょう。



⑥ライセンス追加：本リポジトリをOSS(オープンソースソフトウェア)にする場合、「ライセンス」を記載するかと思います。そのテンプレートを作成するかどうかです。私はOSSで公開したことがないため、下記項目は分かりません。必要な際は調べてみてください🙏

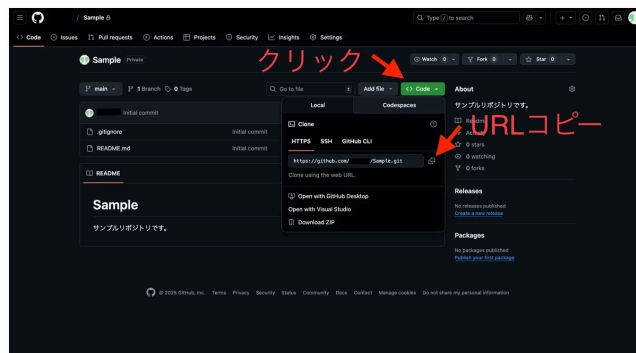


⑦以上で設定できた場合はCreateRepositoryをクリック。リポジトリが下記のように作成されます。



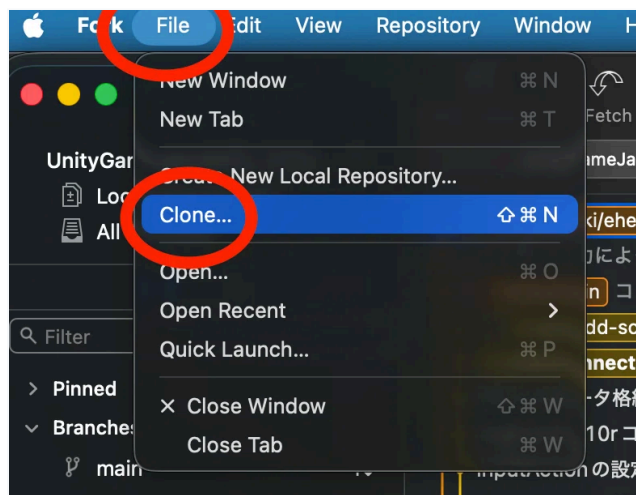
入力した情報が反映されていますね(´▽｀)

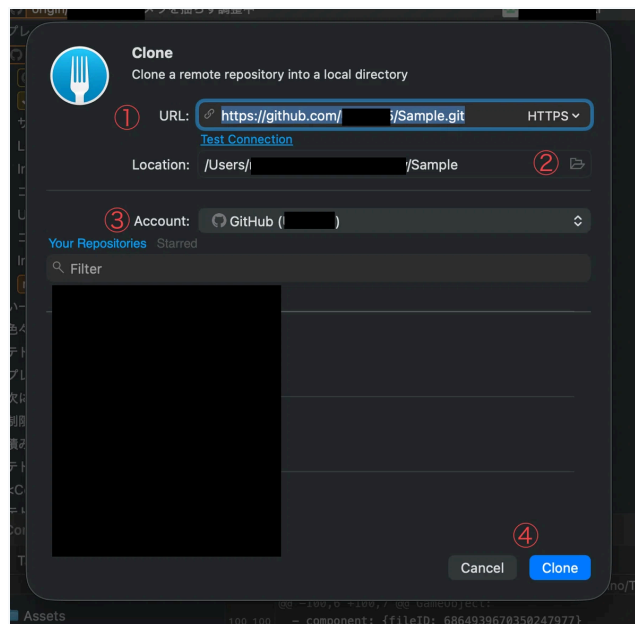
2. リポジトリURLをコピー



今回は導入が簡単なHTTPSにしていますが、
長期運用を考えるならSSHが良い。安全性も高い。ただし設定が複雑。
(GitHubCLIはコマンドラインツールです)

3. Forkでリポジトリをクローン

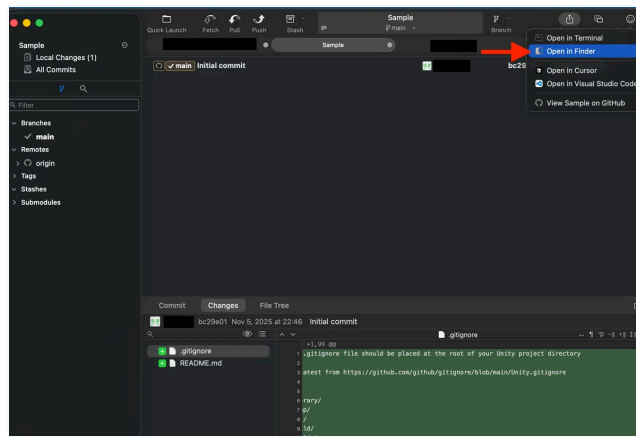




- ①リポジトリURLです。先ほどコピーしていたURL（HTTPS）が入っているはず。
- ②クローンするリポジトリの格納場所。フォルダアイコンからパスを選択しましょう。パスに日本語が入っているのは避けたいですね。
- ③アカウント。自身のアカウントを選びましょう。GitHubと連携させているはず！
- ④以上で設定できた場合はCloneをクリック。

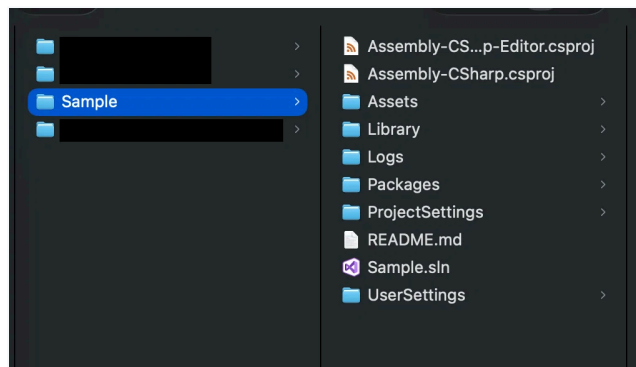


指定していたパスにリポジトリが生成されていることを確認しましょうd(￣ー￣)

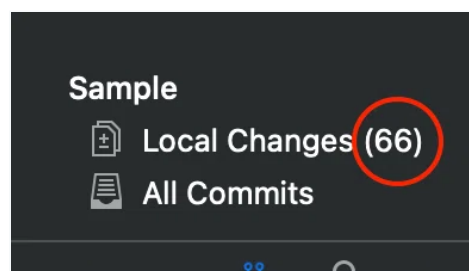


ここからならリポジトリのフォルダすぐ開けます👁️

4. Unityプロジェクトをリポジトリ直下に入れましょう。
※Unityプロジェクトを作っている状態前提で話を進めます。

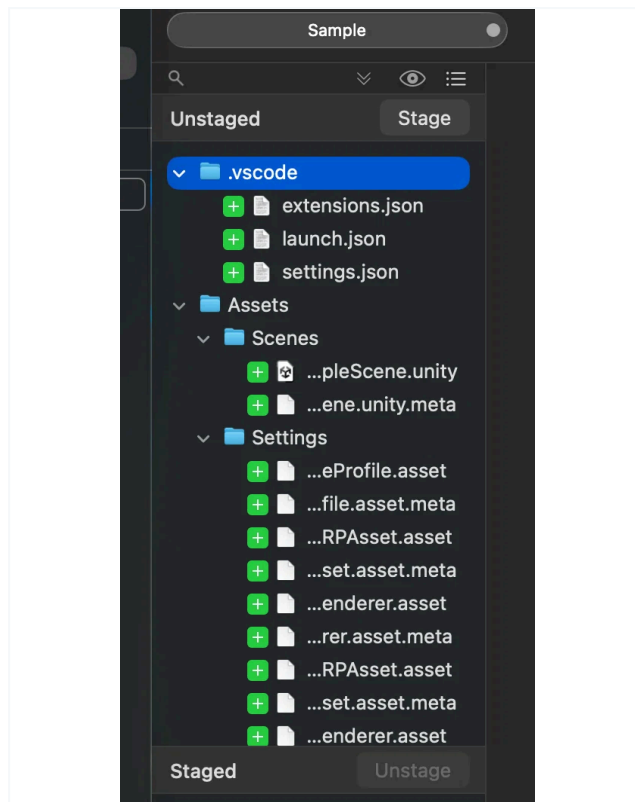


5. 変更ファイルを確認しよう。
Forkの画面左上、LocalChangesに数字が出てきているのがわかるかと思います。

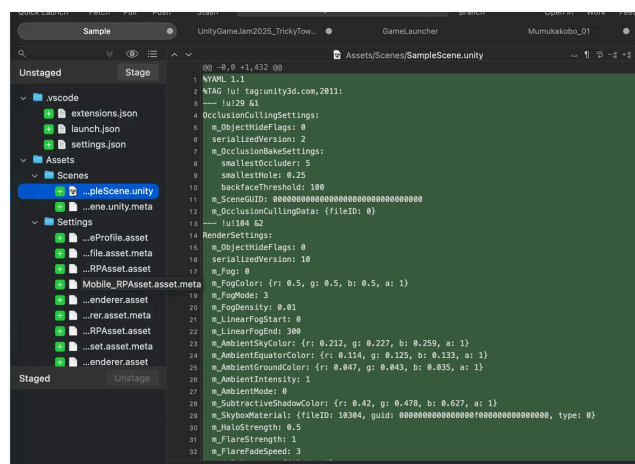


LocalChanges：ローカルPC（リポジトリ内）の変更の数。ファイルの追加/削除、変更を行った際に表示されます。私の場合66件出ていますね。

ここを押すと、「Unstaged」に追加されたファイルが表示されているかと思います。



今回はUnityのデーター式が入っていますね。



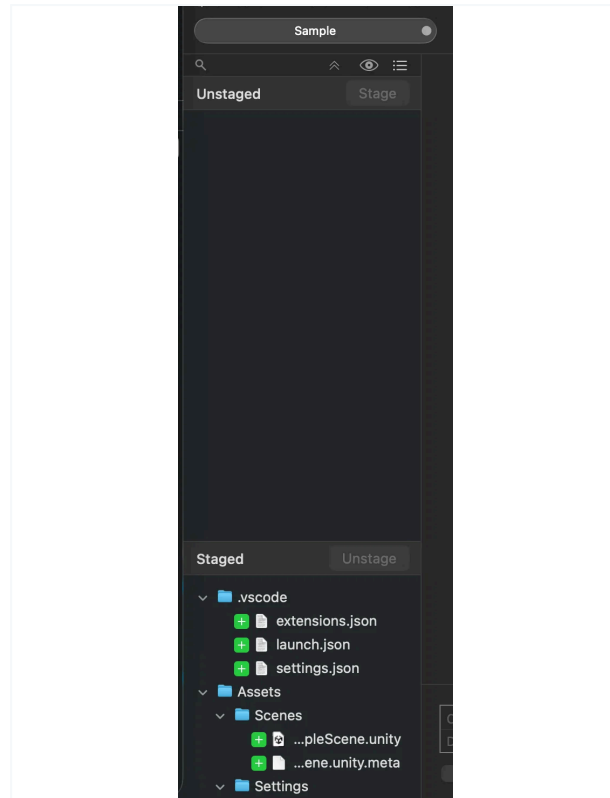
試しに.unityファイルを押してみると、データが確認できます。

Unstaged・・・コミットの準備ができていない変更ファイルたち。まだ作業中で、コミット対象ではない状態。

6. ステージング

ステージング・・・選択したファイルをコミット対象リスト（Staged）に移動させる。

Unstagedに表示されたファイルたちを全選択（command+A）し、Stageボタンを押下。
（command+control+S）



✅ UnstagedとStagedを分けれることによって、このファイルはコミットしたいけど、このファイルはまだコミットしたくない...といったときに活躍します。
ゲーム作ってたらこのような場面出てくるので、
そのとき分かればOK。d(――)

7. コミット

Stagedの右側にコミット操作の欄があります。

Commit subject：コミットの主題。どういう実装をしたかなどを記載。
Description：説明。私は基本書かない。

これらを記入しコミットボタンを押下。

⚠ 本来はそのままmainブランチにプッシュせず、先にブランチ作成してから作成したブランチにコミットしますが、今回は割愛。



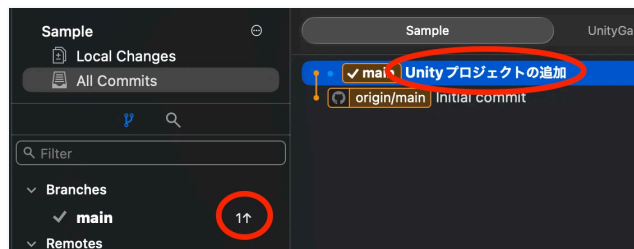
Tips

Amendは「修正する」です。これは、直前のコミットを修正（上書き）する機能です。

【使用用途】

例えば、コミットして「あ！このファイル入れ忘れてた！」「コミットの名前タイプミスしちゃってる💧」といった際に使えたりします。

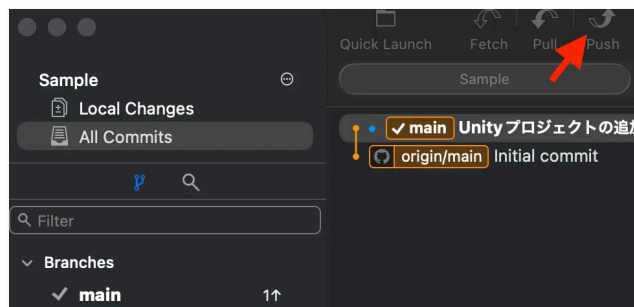
⚠️ Push後の場合履歴が書き変わるので、他人との履歴がずれて「コンフリクト」が起こる可能性がある。扱いには要注意。



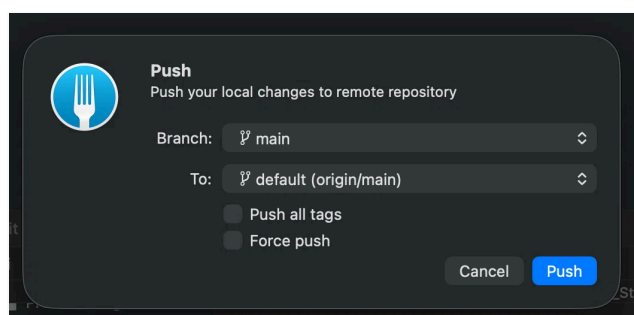
AllCommits：すべてのコミット履歴

mainの「1↑」は、1件のコミットをGitHubのアップロード（Push）出来ますという意。

8. GitHubにプッシュ



ここ押すだけでOK



Branch：どのブランチの内容を
To：どのブランチにPushするのか

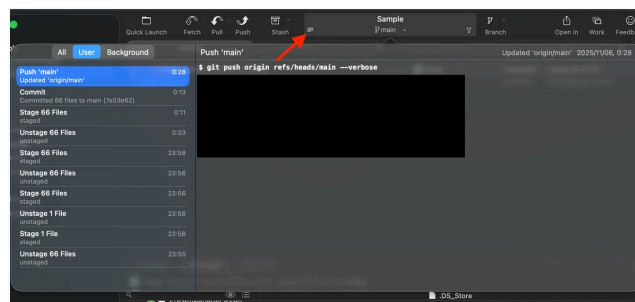
Push all tags : タグも送るか (offでOK)
Force push : 履歴を強制上書き (マジでやめてね)

以上で良さそうだったらPushボタン押下。

Tips

Pushする際なかなかPushが終わらなかったりする時があります。
大体はネットワーク問題。送信可能なデータ量を超していた場合はエラーメッセージのダイアログがポップされる。
ただ、闇雲に待つのも気が引けるので、操作状況を下記画像の矢印から表示できます。ここで今何を行っているかを見てみるのも良いかもしれません。

また、**1ファイル50MBを超えるファイルをアップロードしようとするときエラー吐きます**。リポジトリの肥大化を防ぐため制限が設けられているからです。画像ファイルなどの場合は圧縮して対応しましょう。



9. GitHub上にUnityプロジェクトがアップロードされた。Let's確認👀
これでチームメンバーに同じUnityプロジェクトを共有できる状態になります。

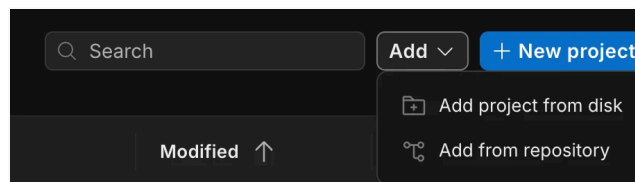


 チームメンバーがプロジェクトを取得する
(Clone)

GitHubで共有したリポジトリは、チームメンバーが自分のPCにコピーして使えます。
Forkを開いて「File/Clone...」を選び、GitHubのURLを入力するだけです。

Cloneの流れ:

1. GitHubページで「Code」 → 「HTTPS」をコピー
2. Fork → 「File/Clone...」を押下
3. 保存先を指定してClone実行
4. UnityHub → 「Add/Add project from disk」からリポジトリ（Unityプロジェクト）を開く

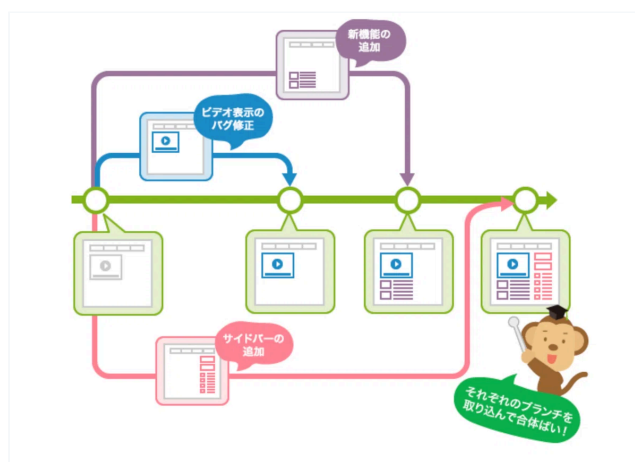


⚠ 全員が同じUnityバージョンを使いましょう。

⚠ 作業を始める前に「Fetch」「Pull」で最新データを取得するのも大切です。

🌿 ブランチを使って安全に作業する

同じプロジェクトを複数人が編集する場合、直接mainブランチを変更すると衝突しやすくなります。
安全に作業するには、『ブランチ（枝）』を分けるのが基本です。



<https://backlog.com/ja/git-tutorial/stepup/01/>
こちらから引用。とても図解が分かりやすかった👍

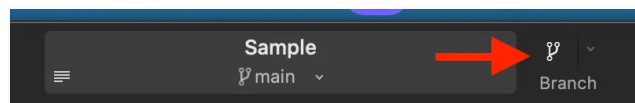
ブランチ作成の流れ:

1. Forkで「New Branch」ボタンをクリック

mainを押下すると「New Branch...」が出てくるのでそちらをクリック。



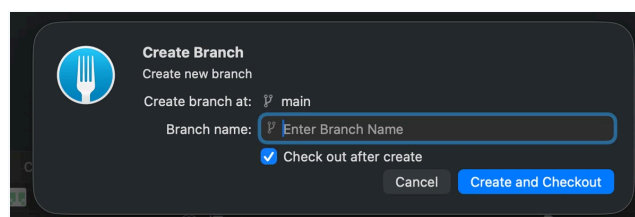
もしくはこのボタンを押下



ブランチ名を付ける。

⚠ 日本語はやめましょう。ローマ字もやめましょう。

(例：feature/名字/playerMove)



Check out after create :

ブランチを作成したら作成したブランチに自動でチェックアウトする機能。

これはonにしておいた方がトラブル減るのでオススメd(￣ー￣)

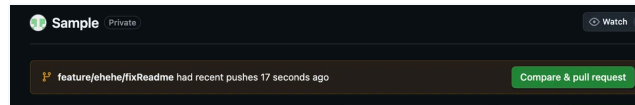
あとはUnityプロジェクトをコミット→プッシュ→...した時と流れは同様。

これで自身が作成した実装を反映させることができますが、mainにまだ実装を反映できていない状態です。

最後に、Pull Requestを送ってマージさせてみましょう！

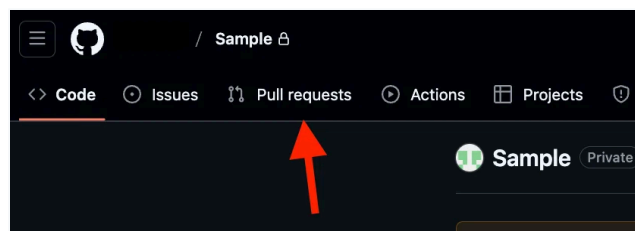
🤖 ブランチ内容をmainに合体(マージ)させる流れ:

GitHubに行くとこのようにpull request送れるよ〜ボタンが出てきます。出てきていたら押下。



Pull Request・・・このブランチのデータ、そのブランチに合体（マージ）させてもいいですか？の申請だと思ってください。

出てきていない場合はこちらからプルリクエストを送りましょう。

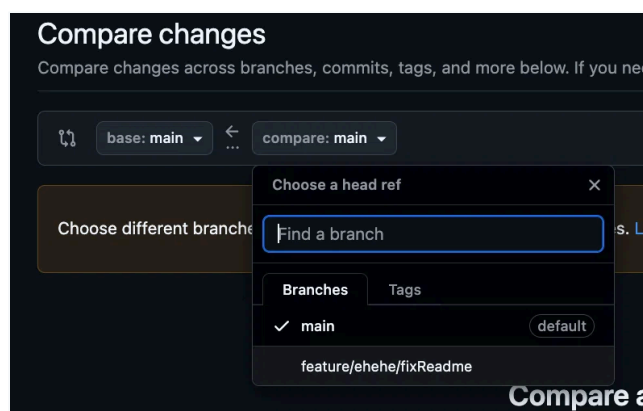


base:main ← compare:main

左側が反映先、右側が反映するデータ。今回だとmainにfixReadmeブランチの内容を反映させたいため、

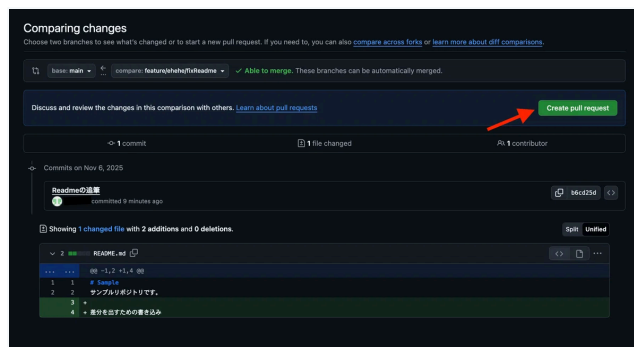
base:main ← compare:feature/ehehe/fixReadme

のように変更しましょう。



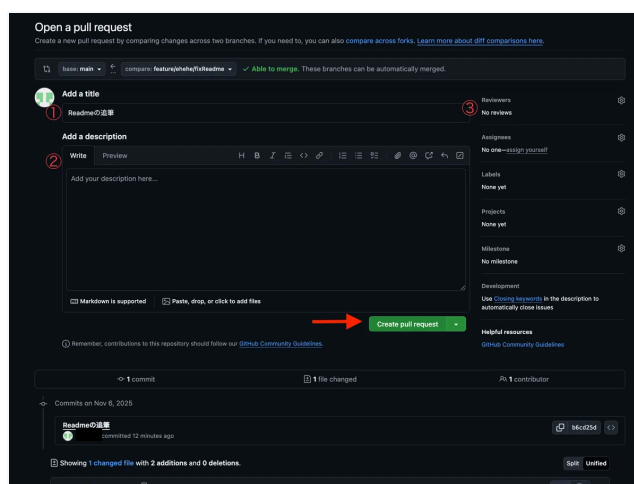
Create pull requestを押下。

「☒ Able to merge.」となっていなかったらどこかでコンフリクト（競合）が起きています。修正しましょう。



commit : コミット数
file changed : 変更ファイル数
contributor : コミットした人数

Create pull requestを押すとプルリクエストが送れます。

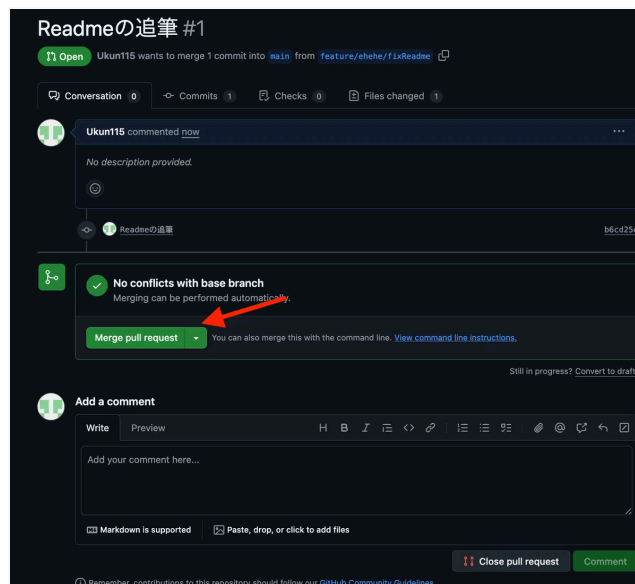


- ① リクエストの主題。リクエストを見てもらう相手に伝わるタイトルにしましょう。
- ② 内容。markdown形式で書けます。動画や画像も添付できます。Previewに切り替えることでどう見えるかを確認することもできます。
- ③ その他各種設定。下記画像にて解説。

項目	意味	使いどころ
Reviewers	レビュー担当者	コード確認や承認を依頼する
Assignees	担当者	誰が責任を持つか明確に
Labels	ラベル (タグ)	種類・目的ごとに整理
Projects	プロジェクトボード	チームの進捗管理に利用
Milestone	マイルストーン	リリースやスプリント単位で管理
Development	開発連携 (自動クローズ等)	Issue とのリンク設定
Helpful resources	参考リンク	GitHub ガイドラインなど

これらは設定しなくてもOK。プロジェクトに合わせてみましょう。

第三者に「変更ファイル」などの内容を確認してもらい、OKだったらここを押下。



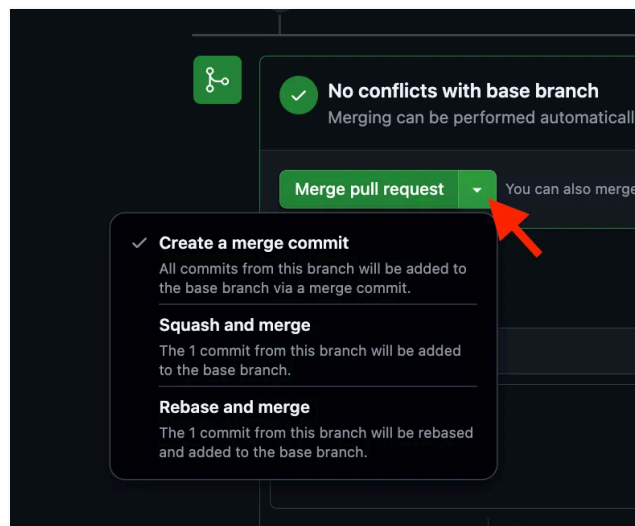
変更ファイルの確認は、「Files changed」から確認できます。



!! この確認はとても重要です。
めんどくさがるず、必ず確認しましょう！

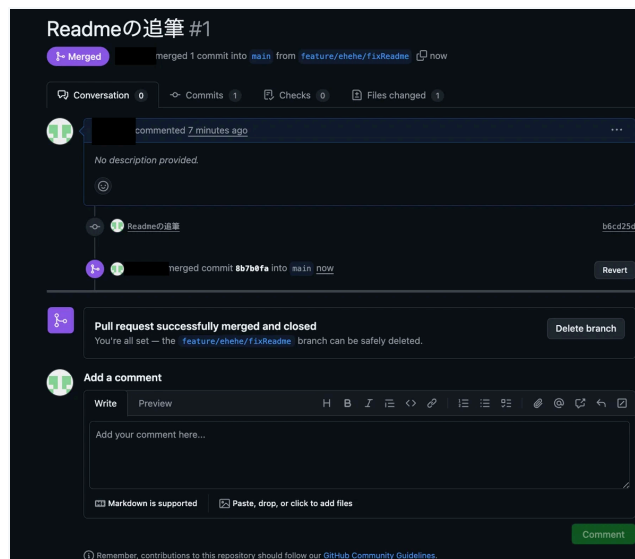
マージする際、「Squash and merge」がおすすめ。

マージ出来ると、コミットメッセージを1つに集約できます。Fork上の視認性が上がるため好き👍



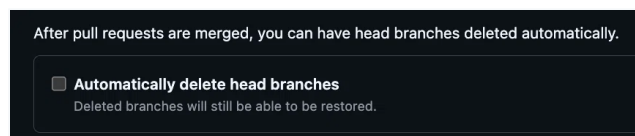
マージ完了です👏

今回マージさせたブランチは大体使わなくなるため、「Delete branch」を押下し、ブランチを削除してください。



Tips

設定/一般からマージ後自動でブランチを削除させる設定もできるため、設定してみてください。



最後に、Fork上で「Fetch」「(main) をPull」してください。そうするとmainを今回の内容を合体させた新しいデータにすることができます👏

あとはこの繰り返し...🔄

⚠️ コンフリクト(競合)をForkで解決する

シーンやPrefab、スクリプトを複数人が同時に編集すると、競合（Conflict）が発生することがあります。

Forkでは、マージ時に警告が出たら「Resolve Conflicts」を選び、GUIで解決可能です。

解決の流れ：

1. Forkが競合箇所を検出
2. 左右の変更を見比べて残す方を選択
3. 保存 → コミット

⚠️ Fork上では解決できないコンフリクトも起きたりします。その場合はGitコマンドで解決しないといけなかったり、そもそもそうならないようにコミットしたりといった別途対応が必要になります。



チーム運用のベストプラクティス

開発をスムーズに進めるために、チーム内で以下のルールを決めましょう。

✅ ブランチ命名ルール

どういう種類のブランチ？誰のブランチ？とかは入れておいた方がいいかも

feature/tarou/〇〇：新機能
fix/tarou/〇〇：修正
hotfix/tarou/〇〇：緊急対応

✅ コミットメッセージ例

初めに動詞入れておくと良い。

ただ、日本語でもOKなので、日本語で書きちゃいましょう。

- Add player jump animation
- Fix enemy collision bug

✅ 作業ルール

こういったルールはメンバー間で擦り合わせておいた方が良い。

- 新規ブランチを作る際は最新のmainブランチから伸ばす
- 作業後に「Commit/Push」を徹底
- マージは必ず第三者が承認する



まとめ

Forkを使えば、コマンド操作なしでGitを完全に扱えるようになります。

便利ですね👍

GitHubと組み合わせることで、チームメンバー全員が安全・効率的にUnity開発を進められます。

次は、Pull Requestを使ったコードレビューや、GitHub Actionsによる自動ビルドについて理解を深めても良いかもしれません。

このワークフローを身につければ、チームのデータ共有はとてもスムーズに行くはずです！
ファイト🔥